

Lesson 03: Repeating Counter

You have seen a laptop program print words and use a typed name. Now you will watch a stored number change. First, one program prints 1 through 5 and stops. Then the final program starts at 1 again and keeps counting until you stop it.

Learning Goals

By the end of this lesson, you will be able to:

- describe `count` as a counter whose stored number changes;
- identify a `for` loop that counts from 1 through 5;
- explain the difference between a program that stops after 5 and one that repeats;
- connect the one-second wait to the visible pause; and
- stop a repeating terminal program with Ctrl+C.

Before You Begin

Complete [lesson 02: Name Greeting](#). Use the same teacher-prepared Linux laptop, VS Code, terminal, and GCC compiler. You do not need an Arduino.

Core Mission: Count Once

Goal: Watch five numbers appear on separate lines, about one second apart, and see the terminal prompt return by itself.

Prepare: Use the supplied `count-once.c` without editing it. This first program stops on its own.

1. In VS Code, choose **File > Open Folder...** and select `coding-fundamentals-lessons/03-repeating-counter`.
2. Choose **Terminal > New Terminal**.
3. Check the current folder:

```
pwd
```

Its path must end with `coding-fundamentals-lessons/03-repeating-counter`.

4. List the files:

```
ls
```

Confirm that you see both `count-once.c` and `main.c`.

5. Build the finite program from `count-once.c`:

```
gcc -std=c11 -Wall -Wextra -Werror count-once.c -o count-once
```

A successful build usually returns to the prompt without printing a message.

6. Run it:

```
./count-once
```

Notice: Write the numbers you saw in order:

About how long was the pause between numbers? _____

Did the terminal prompt return after 5 without Ctrl+C? **Yes / No**

Stop when the prompt returns. You have completed the first mission; leave the source files unchanged.

Name the Counter and Finite Loop

Open `count-once.c`. Its focus is the changing counter inside a loop:

```
for (int count = 1; count <= 5; count++) {  
    printf("%d\n", count);  
    fflush(stdout);  
    sleep(1);  
}
```

`count` is a stored number. The `for` loop sets it to 1, then changes it to 2, 3, 4, and 5. `printf` prints the current number. `sleep(1)` waits about one second before the next number. After the fifth turn, this loop finishes and the program returns to the prompt.

`fflush(stdout)` is a helper that makes each number appear before the wait. You do not need to explain its punctuation. The one-second `sleep()` instruction comes from the prepared Linux environment; it is not a timing instruction for every kind of C computer.

Compare: What If the Whole Count Repeats?

The final supplied program in `main.c` keeps the same 1–5 counting loop and puts it inside an outer `while` loop. You do not need to edit either source file.

Before running a program that repeats, find the **Ctrl** and **C** keys. Hold Ctrl and press C once to ask the program to stop. The prompt should return; you may see `^C`.

1. **Predict:** Write the first seven numbers you expect if the program restarts at 1 after 5.

Circle where the second count begins.

2. Build the final repeating program from the unchanged `main.c`:

```
gcc -std=c11 -Wall -Wextra -Werror main.c -o repeating-counter
```

3. Run it:

```
./repeating-counter
```

4. Watch until you have seen at least `1, 2, 3, 4, 5, 1`. Then hold **Ctrl** and press **C** once. Do not wait for this program to return to the prompt by itself.

Notice: Write the first six numbers you actually saw:

Did 1 appear again after 5? **Yes / No**

Did the prompt return after you pressed Ctrl+C? **Yes / No**

Stop with the prompt visible. Both runs are complete.

Explain the Two Loops

Open `main.c` after you have observed the restart. The final source is:

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    while (1) {
        for (int count = 1; count <= 5; count++) {
            printf("%d\n", count);
            fflush(stdout);
            sleep(1);
        }
    }
}
```

The inner `for` loop changes `count` from 1 through 5. The outer `while (1)` loop starts the whole 1–5 count again after 5:

```
finite program:    1 → 2 → 3 → 4 → 5 → prompt
repeating program: 1 → 2 → 3 → 4 → 5 → 1 → 2 → ...
```

Complete these short explanations:

- The stored value named `count` changed from ____ through ____.
- The `for` loop _____.
- The outer `while` loop _____.
- My prediction was the same as / different from the observed restart because _____.

The repeating program did not freeze. Repeating was one of its instructions, and Ctrl+C was the planned stop.

Optional Cleanup

These are two different generated executables. To remove them, use their exact names, one command at a time:

```
rm count-once
```

```
rm repeating-counter
```

Do not include `count-once.c` or `main.c` in either cleanup command. You can build both executables again from the source files.

Troubleshooting

- **`count-once.c` or `main.c` does not appear after `ls`:** Reopen the `03-repeating-counter` folder.
- **GCC is missing or reports an error:** Ask the teacher to check the workstation or read the first compiler error with you.
- **An executable cannot be found:** Confirm the matching build finished, then use `./count-once` or `./repeating-counter` from this folder.
- **All numbers seem to appear together:** Confirm the supplied code has `fflush(stdout) ;` before `sleep(1) ;`, rebuild, and try again.
- **The finite program did not return after 5:** Make sure you ran `./count-once` rather than `./repeating-counter`.
- **The repeating program does not return by itself:** That is expected. Hold Ctrl and press C once.
- **You typed the letter C but the program continued:** Hold the Ctrl key while pressing C.

Next, use the same counting idea to control a physical light in [lesson 04: Arduino LED Counting](#).